Week2_Script_OPenAI_1__2__1

June 8, 2026

0.1 Securely Storing OpenAI API Key with Colab Secrets

To securely store your OpenAI API key and prevent it from being exposed in your notebook or source code, you can use Colab Secrets. Here's how:

1. **Open Colab Secrets**: Click the " " (Secrets) icon in the left-hand panel of your Colab notebook.
2. **Add a New Secret**: Click the "+ New secret" button.
3. **Name the Secret**: For the 'Name' field, enter `OPENAI_API_KEY` (or any other name you prefer).
4. **Enter Your API Key**: For the 'Value' field, paste your actual OpenAI API key.
5. **Save Secret**: Click 'Save'.
6. **Enable Notebook Access**: Make sure the toggle switch next to your `OPENAI_API_KEY` secret is turned ON for the current notebook. This allows your notebook to access the secret.

[57]:
```python
# Import the necessary library to access Colab secrets
from google.colab import userdata
from openai import OpenAI

# Retrieve your API key from Colab Secrets
# The name 'OPENAI_API_KEY' should match the name you gave your secret in the␣
 ↪Colab Secrets panel.
OPENAI_API_KEY = userdata.get('OPENAI_API_KEY')

# Initialize the OpenAI client with the securely retrieved API key
client = OpenAI(api_key=OPENAI_API_KEY)

# Example usage (you can replace this with your specific API call)
# completion = client.chat.completions.create(
#     model="gpt-4.1",
#     messages=[
#         {
#             "role": "user",
#             "content": "Tell me a fun fact about Python programming."
#         }
#     ]
# )
# print(completion.choices[0].message.content)
```

```
print("OpenAI client initialized successfully using API key from Colab Secrets.
  ↪")
```

```
OpenAI client initialized successfully using API key from Colab Secrets.
```

0.1.1 List all accessible OpenAI models

This code snippet uses the `client.models.list()` method to retrieve a list of all models that your current API key has access to. It then iterates through the list and prints the ID of each model. This can help you confirm whether DALL-E models (like `dall-e-2` or `dall-e-3`) are indeed available to your account.

[79]:
```
# List all accessible models
all_models = client.models.list()

print("Accessible models (IDs only):")
for model in all_models.data:
    print(model.id)
```

```
Accessible models (IDs only):
text-embedding-ada-002
whisper-1
gpt-3.5-turbo
tts-1
gpt-3.5-turbo-16k
gpt-4-0613
gpt-4
davinci-002
babbage-002
gpt-3.5-turbo-instruct
gpt-3.5-turbo-instruct-0914
gpt-3.5-turbo-1106
tts-1-hd
tts-1-1106
tts-1-hd-1106
text-embedding-3-small
text-embedding-3-large
gpt-3.5-turbo-0125
gpt-4-turbo
gpt-4-turbo-2024-04-09
gpt-4o
gpt-4o-2024-05-13
gpt-4o-mini-2024-07-18
gpt-4o-mini
gpt-4o-2024-08-06
omni-moderation-latest
omni-moderation-2024-09-26
o1-2024-12-17
o1
```

```
o3-mini
o3-mini-2025-01-31
gpt-4o-2024-11-20
gpt-4o-mini-search-preview-2025-03-11
gpt-4o-mini-search-preview
gpt-4o-transcribe
gpt-4o-mini-transcribe
o1-pro-2025-03-19
o1-pro
gpt-4o-mini-tts
o3-2025-04-16
o4-mini-2025-04-16
o3
o4-mini
gpt-4.1-2025-04-14
gpt-4.1
gpt-4.1-mini-2025-04-14
gpt-4.1-mini
gpt-4.1-nano-2025-04-14
gpt-4.1-nano
gpt-image-1
gpt-4o-transcribe-diarize
gpt-5-chat-latest
gpt-5-2025-08-07
gpt-5
gpt-5-mini-2025-08-07
gpt-5-mini
gpt-5-nano-2025-08-07
gpt-5-nano
gpt-audio-2025-08-28
gpt-realtime
gpt-realtime-2025-08-28
gpt-audio
gpt-5-codex
gpt-image-1-mini
gpt-5-pro-2025-10-06
gpt-5-pro
gpt-audio-mini
gpt-audio-mini-2025-10-06
gpt-5-search-api
gpt-realtime-mini
gpt-realtime-mini-2025-10-06
sora-2
sora-2-pro
gpt-5-search-api-2025-10-14
gpt-5.1-chat-latest
gpt-5.1-2025-11-13
gpt-5.1
```

```
gpt-5.1-codex
gpt-5.1-codex-mini
gpt-5.1-codex-max
gpt-image-1.5
gpt-5.2-2025-12-11
gpt-5.2
gpt-5.2-pro-2025-12-11
gpt-5.2-pro
gpt-5.2-chat-latest
gpt-4o-mini-transcribe-2025-12-15
gpt-4o-mini-transcribe-2025-03-20
gpt-4o-mini-tts-2025-03-20
gpt-4o-mini-tts-2025-12-15
gpt-realtime-mini-2025-12-15
gpt-audio-mini-2025-12-15
chatgpt-image-latest
gpt-5.2-codex
gpt-5.3-codex
gpt-realtime-1.5
gpt-audio-1.5
gpt-4o-search-preview
gpt-4o-search-preview-2025-03-11
gpt-5.3-chat-latest
gpt-5.4-2026-03-05
gpt-5.4-pro
gpt-5.4-pro-2026-03-05
gpt-5.4
gpt-5.4-nano-2026-03-17
gpt-5.4-nano
gpt-5.4-mini-2026-03-17
gpt-5.4-mini
gpt-image-2
gpt-image-2-2026-04-21
gpt-5.5
gpt-5.5-2026-04-23
gpt-5.5-pro
gpt-5.5-pro-2026-04-23
chat-latest
gpt-realtime-translate
gpt-realtime-2
gpt-realtime-whisper
```

1 *** Unit 3 Lecture***

2 *This script will be a part of HW #2 Functions and Modules*

Slide 1: Introduction to Functions

[1]:
```
print("Functions allow us to reuse code and make our programs more modular.")
```

```
Functions allow us to reuse code and make our programs more modular.
```

Slide 2-3: Defining and Calling a Simple Function

[2]:
```
def greet():
    print("Hello! Welcome to Python Functions.")

# Run the function
greet()
```

```
Hello! Welcome to Python Functions.
```

Challenge 1

[3]:
```
# Challenge 1: Define a function that prints "Python is fun!"
# Complete the code below
def python_fun():
    print("Python is fun!")
```

Slides 6-7: Function with Parameters

[4]:
```
def personalized_greeting(name):
    print(f"Hello, {name}! Welcome to Python Functions.")

# Run the function
personalized_greeting("Alice")
```

```
Hello, Alice! Welcome to Python Functions.
```

Challenge 2: Define a function that takes two arguments and prints their sum

[5]:
```
# Complete the code below
def print_sum(a, b):
    print(a + b)
```

Slide 7: Returning Values from Functions

[6]:
```
def add(a, b):
    return a + b

# Run the function and print the result
result = add(3, 5)
print(f"The sum is: {result}")
```

```
The sum is: 8
```

Challenge 3: Define a function that returns the product of two numbers

[7]:
```
# Complete the code below
def multiply(a, b):
```

```
    return a * b
```

Slide 8-9: Local and Global Variables

[8]:
```
# Example 4: Local Variable
def local_variable_example():
    local_var = 10
    print(f"Local variable: {local_var}")

# Run the function
local_variable_example()
```

```
Local variable: 10
```

Challenge 4: Define a function that declares a local variable and prints it

[9]:
```
# Complete the code below
def print_local():
    local_var = 42
    print(f"Local variable: {local_var}")
```

Slide 10: Global Variable

[10]:
```
global_var = 20

def global_variable_example():
    print(f"Global variable: {global_var}")

# Run the function
global_variable_example()
```

```
Global variable: 20
```

Challenge 5: Modify the global variable within a function

[11]:
```
# Complete the code below
global_var = 20  # keep this the same variable as above

def modify_global():
    global global_var
    global_var = global_var + 15   # or any change you like
```

Slide 11-13: Passing Arguments to Functions

[12]:
```
# Example 6: Passing Multiple Arguments
def print_full_name(first_name, last_name):
    print(f"Full name: {first_name} {last_name}")

# Run the function
print_full_name("John", "Doe")
```

```
Full name: John Doe
```

Challenge 6: Define a function that takes three arguments and prints them

[13]:
```python
# Complete the code below
def print_three(a, b, c):
    print(a, b, c)
```

Slide 14-15: Keyword Arguments

[14]:
```python
def describe_pet(animal_type, pet_name):
    print(f"\nI have a {animal_type}.")
    print(f"My {animal_type}'s name is {pet_name}.")

# Run the function using keyword arguments
describe_pet(animal_type='hamster', pet_name='Harry')
```

```
I have a hamster.
My hamster's name is Harry.
```

Challenge 7: Define a function with keyword arguments and call it

[15]:
```python
# Complete the code below
def show_vehicle(type, brand):
    print(f"Vehicle type: {type}")
    print(f"Brand: {brand}")

# example call
show_vehicle(type="car", brand="Toyota")
```

```
Vehicle type: car
Brand: Toyota
```

Slide 10-12: Default Argument Values

[16]:
```python
# Example 8: Default Argument
def print_greeting(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

# Run the function with and without the default argument
print_greeting("Alice")
print_greeting("Bob", "Hi")
```

```
Hello, Alice!
Hi, Bob!
```

Challenge 8: Define a function with a default argument and call it

[17]:
```python
# Complete the code below
def introduce(name, age=25):
    print(f"My name is {name} and I am {age} years old.")
```

```
# example calls
introduce("Alice")
introduce("Bob", 30)
```

```
My name is Alice and I am 25 years old.
My name is Bob and I am 30 years old.
```

Slide 13: Value-Returning Functions

[18]:
```
# Example 9: Value-Returning Function
def square(number):
    return number * number

# Run the function and print the result
result = square(4)
print(f"The square of 4 is: {result}")
```

```
The square of 4 is: 16
```

Challenge 9: Define a function that returns the cube of a number

[19]:
```
# Complete the code below
def cube(number):
    return number ** 3
```

Slides 14-16: The random and math Modules

[20]:
```
import random
# Generate a random float in the range [0.0, 1.0)
number = random.random()
print("Random float between 0.0 and 1.0:", number)
```

```
Random float between 0.0 and 1.0: 0.890469163624902
```

[21]:
```
import random
# Generate a random integer between 1 and 100
number = random.randint(1, 100)
print("Random number between 1 and 100:", number)
```

```
Random number between 1 and 100: 24
```

[22]:
```
import math

# Example 10: Using the math Module
def calculate_circle_area(radius):
    return math.pi * radius * radius

# Run the function and print the result
area = calculate_circle_area(5)
print(f"The area of the circle is: {area:.2f}")
```

```
The area of the circle is: 78.54
```

Challenge 10: Define a function that uses the math module to calculate the square root of a number

[23]:
```python
# Complete the code below
import math

def calculate_sqrt(number):
    return math.sqrt(number)
```

Slide 20-21: Storing Functions in Modules

[24]:
```python
# Example 11: Creating and Using Modules
# Create a file named my_module.py with the following content:
# def my_function():
#     print("Hello from my_module!")

# from my_module import my_function
# my_function()
#  Using Functions from a Module in Different Files
# Create another file named main.py with the following content:
# from my_module import my_function

# my_function()

%%writefile custom_module.py
def function_one():
    print("Function One")

def function_two():
    print("Function Two")
my_function()
```

```
Writing custom_module.py
```

uncommend 2 lines below

[27]:
```python
%%writefile my_module.py
def my_function():
     print("Hello from my_module!")
```

```
Writing my_module.py
```

[28]:
```python
from my_module import my_function
my_function()
```

```
Hello from my_module!
```

Challenge 11: Create your own module with at least two functions and import it here

[29]:
```
# Complete the code below be creating with your functions
# Create a file named custom_module.py with the following content:
# def function_one():
#     print("Function One")
# def function_two():
#     print("Function Two")

# Import and use the functions from custom_module
# from custom_module import function_one, function_two
# function_one()
# function_two()

%%writefile custom_module.py
def function_one():
    print("Function One")

def function_two():
    print("Function Two")


from custom_module import function_one, function_two

function_one()
function_two()
```

```
Overwriting custom_module.py
```

[31]:
```
# Example 14: Generating Random Numbers
import random

def roll_die():
    return random.randint(1, 6)

# Run the function and print the result
die_roll = roll_die()
print(f"Rolled a die: {die_roll}")
```

```
Rolled a die: 2
```

Challenge 13: Define a function that generates a random float between two numbers

[32]:
```
# Complete the code below
import random

def random_float(min_value, max_value):
    return random.uniform(min_value, max_value)

print(random_float(1.5, 5.5))
```

```
3.514087322993416
```

[33]:
```python
# Example 15: Returning Multiple Values
def min_max(numbers):
    return min(numbers), max(numbers)

# Run the function and print the result
min_val, max_val = min_max([1, 2, 3, 4, 5])
print(f"Min: {min_val}, Max: {max_val}")
```

```
Min: 1, Max: 5
```

Challenge 14: Define a function that returns the sum, difference, and product of two numbers

[34]:
```python
# Complete the code below
def calculate(a, b):
    sum_val = a + b
    diff_val = a - b
    prod_val = a * b
    return sum_val, diff_val, prod_val

# Example call to test it:
s, d, p = calculate(10, 5)
print(f"Sum: {s}, Difference: {d}, Product: {p}")
```

```
Sum: 15, Difference: 5, Product: 50
```

Returning None from a Function

[35]:
```python
def divide(num1, num2):
    if num2 == 0:
        return None
    else:
        return num1 / num2

# Run the function and handle the result
result = divide(10, 0)
if result is None:
    print("Cannot divide by zero!")
else:
    print(f"Result: {result}")
```

```
Cannot divide by zero!
```

Challenge 15: Define a function that returns None if a number is negative

[36]:
```python
# Complete the code below
def check_positive(number):
     if(number>0):
      return True
     else:
```

```
      return None

print(check_positive(5))
print(check_positive(-3))
```

```
True
None
```

Conditional Execution of the main Function

[37]:
```
def main():
    print("This is the main function.")

def secondary_function():
    print("This is a secondary function.")

if __name__ == '__main__':
    main()
```

```
This is the main function.
```

Challenge 16: Define two functions and conditionally execute one based on a variable value

[38]:
```
def function_one():
    print("One")
def function_two():
    print("two")

a=2-1

if(a==1):
      function_one()
else:
      function_two()
```

```
One
```

[39]:
```
%%writefile nasa.py
# apod.py
import requests
from IPython.display import display, Image

def get_apod(api_key):
    response = requests.get(f'https://api.nasa.gov/planetary/apod?
 ↪api_key={api_key}')
    if response.status_code == 200:
        data = response.json()
        return data['title'], data['url'], data['explanation']
    else:
```

```
        return "Could not fetch APOD data", "", ""

def display_apod(api_key):
    title, url, explanation = get_apod(api_key)
    if title == "Could not fetch APOD data":
        print(title)
    else:
        print(f"Title: {title}\nURL: {url}\nExplanation: {explanation}")
        display(Image(url=url, embed=True))
```

```
Writing nasa.py
```

#check the files created

[44]:
```
# Import the apod module
import nasa

# Use the DEMO_KEY provided by NASA
api_key = "DEMO_KEY"  # Or your specific NASA API key string

# Call the function to display APOD data and image
nasa.display_apod(api_key)
```

```
Title: Jupiter and Venus from Earth
URL: https://apod.nasa.gov/apod/image/2606/JupiterPersonVenus_Nikodem_960.jpg
Explanation: It was visible around the world. The sunset conjunction of Jupiter
(left) and Venus (right) in 2012 was visible almost no matter where you lived on
Earth.  Anyone on our planet with a clear western horizon at sunset could see
them. That year, a creative photographer traveled away from the town lights of
Szubin, Poland to photograph  a near closest approach of the two planets. The
bright planets were then separated by only  three degrees and his daughter
struck a humorous pose. A faint red sunset still glowed in the background.
Jupiter and Venus are together again this week after sunset, passing within a
degree of each other about two days from today.
```

Create your own prompts to test ChatGPT in each of the following sections. Ask instructor for api key if the one proivided here does not work

[46]:
```
from openai import OpenAI

#  Hard-coded API key (replace with your own key of you have it)
client =␣
 ↪OpenAI(api_key="sk-proj-9XGP2B1BAmzYhbkvpo6nT92Hi_zGhRrjwAcJgXTv2y45ZmfQgMVkz9AP0ZLQiqD1DXY

completion = client.chat.completions.create(
    model="gpt-4.1",
    messages=[
        {
            "role": "user",
            "content": "Write a one-sentence bedtime story about a unicorn."
        }
    ]
)
```

```
print(completion.choices[0].message.content)
```

```
Under a velvet moon, a gentle unicorn tiptoed through a sparkling forest,
sprinkling dreams of joy upon every sleeping creature.
```

[49]:
```
from openai import OpenAI
# Replace "YOUR_API_KEY" with your actual OpenAI API key or use the one from␣
 ↪the previous cell.
client =␣
 ↪OpenAI(api_key="sk-proj-9XGP2B1BAmzYhbkvpo6nT92Hi_zGhRrjwAcJgXTv2y45ZmfQgMVkz9AP0ZLQiqD1DXY

response = client.responses.create(
    model="gpt-4.1",
    tools=[{"type": "web_search_preview"}],
    input="What was a positive news story from today?"
)

print(response.output_text)
```

```
On June 7, 2026, several uplifting news stories emerged:

- **Bermuda College Nursing Graduates**: Matisse Bascome, Elizabeth Bento, Jade
Smith, Stephanie Fleming, Abriana Thomas, China Nisbett, Faheemah Scraders, and
Christopher Trott completed their associate degrees in nursing at Bermuda
College, marking a significant achievement in their medical careers.
([bernews.com](https://bernews.com/2026/06/sunday-june-7th-good-news-
spotlight-2/?utm_source=openai))

- **BermudAir Expands Flights**: BermudAir announced new flight routes to the
Cayman Islands and Turks & Caicos, enhancing travel options and connectivity in
the region. ([bernews.com](https://bernews.com/2026/06/sunday-june-7th-good-
news-spotlight-2/?utm_source=openai))

- **Gavin Manders' Gold Medal**: Gavin Manders secured a gold medal in the Mixed
Doubles Open at Pickleball Paradise in Trinidad, showcasing his athletic
prowess. ([bernews.com](https://bernews.com/2026/06/sunday-june-7th-good-news-
spotlight-2/?utm_source=openai))

- **Alec Cutler's Sailing Victory**: Alec Cutler won the J/70 World Championship
in France, highlighting his exceptional sailing skills.
([bernews.com](https://bernews.com/2026/06/sunday-june-7th-good-news-
spotlight-2/?utm_source=openai))

- **Curaçao's World Cup Participation**: Curaçao, a small Caribbean nation with
a population of 156,000, made history by participating in the 2026 FIFA World
Cup, bringing joy and pride to its citizens. ([english.elpais.com](https://engli
sh.elpais.com/archive/2026-06-07/?utm_source=openai))
```

These stories reflect positive developments in education, sports, and international representation.

[61]:
```
from google.colab import userdata
```

[69]:
```
import base64
from openai import OpenAI
from google.colab import userdata

# Retrieve the API key from Colab Secrets
api_key = userdata.get('OPENAI_API_KEY')

client = OpenAI(api_key=api_key)

# Step 1: Get the text completion using a multimodal model like gpt-4o
chat_completion = client.chat.completions.create(
    model="gpt-4o", # Using gpt-4o which is multimodal
    messages=[
        {
            "role": "user",
            "content": "Is a golden retriever a good family dog?"
        }
    ]
)

# Extract the text response
text_response = chat_completion.choices[0].message.content
print("Text response from GPT-4o:", text_response)

# Step 2: Convert the text response to speech using the audio.speech API
speech_response = client.audio.speech.create(
    model="tts-1", # Use a dedicated Text-to-Speech model
    voice="alloy",
    input=text_response,
    response_format="wav"
)

# The speech_response.read() method directly returns bytes
wav_bytes = speech_response.read()

with open("dog.wav", "wb") as f:
    f.write(wav_bytes)

print("Audio saved to dog.wav")
```

```
Text response from GPT-4o: Yes, a Golden Retriever is generally considered an
excellent family dog. They are known for their friendly and gentle temperament,
```

```
which makes them great companions for children and adults alike. Golden
Retrievers are also highly intelligent and easy to train, which can be
beneficial in a family setting. Additionally, they tend to get along well with
other pets and are typically very social. However, like any dog, they require
regular exercise, attention, and proper training to ensure they are well-behaved
and happy.
Audio saved to dog.wav
```

#check the files created

[73]:
```python
from openai import OpenAI
from google.colab import userdata

# Retrieve the API key from Colab Secrets
api_key = userdata.get('OPENAI_API_KEY')

client = OpenAI(api_key=api_key)

# The `client.responses.create` method is deprecated and was part of an older␣
 ↪API structure.
# For vision capabilities (analyzing images), you should use `client.chat.
 ↪completions.create`
# with a model like 'gpt-4o' or 'gpt-4-vision-preview' and structure the␣
 ↪messages with image URLs.
# I'll update the code to use the correct method for current OpenAI API for␣
 ↪image analysis.

response = client.chat.completions.create(
    model="gpt-4o", # or "gpt-4-vision-preview"
    messages=[
        {
            "role": "user",
            "content": [
                {"type": "text", "text": "what's in this image?"},
                {
                    "type": "image_url",
                    "image_url": {
                        "url": "https://upload.wikimedia.org/wikipedia/commons/
 ↪d/dd/Gfp-wisconsin-madison-the-nature-boardwalk.jpg", # Changed to a direct␣
 ↪image URL
                        "detail": "low"
                    },
                },
            ],
        }
    ],
)
```

```
print(response.choices[0].message.content)
```

```
This image depicts a scenic landscape with a wooden boardwalk leading through a
grassy meadow. The sky is mostly clear with some scattered clouds, and the area
is surrounded by lush greenery, suggesting a peaceful, natural environment.
```

[85]:
```
from openai import OpenAI
from google.colab import userdata
import os
import requests
from io import BytesIO
from PIL import Image
from IPython.display import display
import base64 # Import base64 for decoding

# Retrieve the API key from Colab Secrets
api_key = userdata.get('OPENAI_API_KEY')

client = OpenAI(api_key=api_key)

try:
    result = client.images.generate(
        model="gpt-image-2-2026-04-21",
        prompt="a white siamese cat",
        size="1024x1024"
    )

    # Check if a URL was successfully generated
    if result and result.data and result.data[0].url:
        image_url = result.data[0].url
        print(image_url)
        resp = requests.get(image_url)
        resp.raise_for_status()

        #    4. Load into PIL
        img = Image.open(BytesIO(resp.content))

        #    5a. If you're in Jupyter, this will display inline
        display(img)

        #    5b. If you're in a plain Python console, this will open it␣
 ↪externally
        img.show()
    # Check if base64 JSON data was generated instead of a URL
    elif result and result.data and result.data[0].b64_json:
        print("Image generated as base64 JSON. Decoding and displaying...")
```

```
        image_data = base64.b64decode(result.data[0].b64_json)
        img = Image.open(BytesIO(image_data))
        display(img)
        img.show() # For external viewer if not in Jupyter
    else:
        print("Error: Image generation did not return a valid URL or base64␣
 ↪JSON. The model might have failed to generate the image.")
        # Print the full result object for more details if needed
        print(result)
except Exception as e:
    print(f"An error occurred during image generation: {e}")
```

```
Image generated as base64 JSON. Decoding and displaying…
```

More on Modules in Class Lab